Assignment 3: Графовые алгоритмы для подготовки фестиваля

Русский перевод предоставлен для понимания задания. Отчёт ANALYSIS.md оформляется на английском языке.

Дисциплина: Design and Analysis of Algorithms

1. Цель и содержание

В задании графовые алгоритмы применяются к трём задачам подготовки университетского фестиваля: соединению площадок, планированию подготовительных работ и выбору маршрутов доставки. Кабельная сеть описывается неориентированным графом; зависимости между работами и маршруты доставки — ориентированными. Каждая часть получает отдельный входной граф.

В предоставленном шаблоне GitHub необходимо реализовать шесть алгоритмов: DSU, Краскала, Прима, поиск SCC методом Косарайю, топологическую сортировку через DFS и кратчайшие пути в ориентированном ациклическом графе (DAG). Выберите один язык: Java, Python, C++ или C#.

Результат работы включает реализации, собственные тесты и анализ в ANALYSIS.md. Основная программа, обработка JSON, примеры и публичные тесты предоставлены в шаблоне.

2. Порядок выполнения

1. Создайте репозиторий через **Use this template → Create a new repository**. Если репозиторий приватный, предоставьте преподавателю доступ.

2. В поле language файла submission.json укажите Java, Python, Cpp или CSharp. Реализуйте шесть методов в соответствующем файле из таблицы.

3. Выполните части 1–5, пройдите публичные тесты и добавьте собственные тестовые случаи.

4. Сравните Краскала и Прима, продемонстрируйте восстановление маршрута и заполните ANALYSIS.md.

5. Сдайте в Moodle ссылку на коммит и успешный запуск Public checks для этого коммита.

Язык	Файл реализации
Java	Java/source/Algorithms.java
Python	Python/source/algorithms.py
C++	Cpp/source/algorithms.hpp
C#	CSharp/source/Algorithms.cs

Реализуйте методы dsu, kruskal, prim, scc, topo и dag. Вспомогательные функции, классы и дополнительные исходные файлы можно размещать непосредственно в папке source выбранного языка. Стандартные коллекции, сортировка и очереди с приоритетом разрешены; требуемые алгоритмы должны быть реализованы студентом самостоятельно.

3. Общие требования

Вершины нумеруются от 0 до n-1. ID ребра — его позиция в массиве edges, начиная с 0. Программа должна сохранять предоставленный JSON-интерфейс, выдавать один результат на каждый запрос и сбрасывать состояние алгоритмов между запросами. Стандартный вывод предназначен только для результатов.

Поддерживайте до 100 000 вершин и 250 000 рёбер или операций DSU. Взвешенные рёбра имеют веса от -1 000 000 000 до 1 000 000 000. Вычисление суммарных стоимостей и расстояний должно безопасно поддерживать знаковые 64-битные целые числа. Реализации DFS должны обрабатывать глубокие графы без переполнения стека вызовов. Точные форматы запросов и ограничения приведены в CONTRACTS.md.

В требованиях к сложности n обозначает число вершин, m — число рёбер, q — число операций DSU. Оценки памяти включают возвращаемый результат.

4. Часть 1: Связность с помощью DSU

Площадки фестиваля соединяются кабелями, которые добавляются по одному. После каждого добавления определите число компонент связности и размер наибольшей компоненты.

Реализуйте dsu на основе системы непересекающихся множеств со сжатием путей и объединением по размеру или рангу. Изначально каждая вершина образует отдельную компоненту. Повторное объединение и объединение вершины с самой собой не изменяют существующую компоненту.

Вход: n и operations — последовательность пар вершин. **Выход:** states — пары [components, largest] после каждой операции. При n=0 массивы operations и states пусты.

Пример: для n=4 и operations=[[0,1],[1,2],[0,2],[3,3]] результат states=[[3,2],[2,3],[2,3],[2,3]]. Последние две операции не объединяют компоненты.

Требуемая сложность: O((n+q) α(n)) амортизированного времени и O(n+q) памяти.

5. Часть 2: Кабельная сеть минимальной стоимости

Для неориентированного взвешенного графа найдите минимальный остовный лес. В каждой компоненте выбранные рёбра должны соединять все вершины без циклов и иметь минимальную суммарную стоимость. Изолированные вершины также считаются компонентами.

Выполните две отдельные реализации: kruskal — с сортировкой рёбер и DSU; prim — со списками смежности и очередью с приоритетом. Прим должен начинать обработку с новой непосещённой вершины, пока не будут обработаны все компоненты.

Вход: n и edges, где ребро представлено как [u,v,weight]. **Выход:** cost, edge_ids и components. Каждый выбранный ID ребра встречается один раз. Параллельные рёбра и отрицательные веса допустимы; петли не включаются в лес. Принимается любой оптимальный лес. При n=0 верните cost=0, edge_ids=[] и components=0.

Пример: n=4, edges=[[0,1,4],[1,2,2],[0,2,3]]. Рёбра с ID [1,2] дают cost=5 и components=2; вершина 3 изолирована. При нескольких оптимальных лесах Краскал и Прим могут вернуть разные допустимые ID рёбер.

Требуемая сложность: Краскал O(n+m log(m+1)); Прим O(n+m log(m+1)) с ленивой кучей или O((n+m) log(n+1)) с индексированной кучей. Память: O(n+m).

6. Часть 3: Компоненты сильной связности

В графе подготовки ребро u→v обозначает зависимость работы v от работы u. Найдите компоненты сильной связности (SCC): максимальные группы вершин, взаимно достижимых по направленным рёбрам.

Реализуйте scc методом Косарайю с двумя обходами DFS и транспонированным графом. Верните метку компоненты для каждой вершины и граф конденсации. Метки могут быть любыми неотрицательными целыми числами. Одинаковая метка должна обозначать ровно одну SCC.

Вход: n и ориентированные рёбра [u,v]. **Выход:** component и condensation. Рёбра конденсации задаются парами меток компонент. Каждая направленная пара разных компонент включается один раз; внутренние рёбра и повторяющиеся пары исключаются. Изолированные вершины образуют отдельные SCC. Повторяющиеся рёбра и петли допустимы. При n=0 оба массива пусты.

Пример: рёбра 0→1, 1→0, 1→2, 0→2, 2→3 допускают component=[5,5,7,9] и condensation=[[5,7],[7,9]]. Два ребра из первой SCC к вершине 2 дают одно ребро конденсации.

Требуемая сложность: O(n+m) ожидаемого времени при удалении повторных пар с помощью хеш-множества; O(n+m) памяти.

7. Часть 4: Порядок подготовительных работ

Для ориентированного графа зависимостей определите, можно ли составить расписание всех работ. Если можно, верните топологический порядок: каждая предварительная работа должна предшествовать зависимой от неё работе.

Реализуйте topo через DFS с тремя состояниями: непосещённая, активная и завершённая вершина. Ребро в активную вершину указывает на цикл. Для ациклического графа обратный порядок завершения DFS даёт требуемый результат.

Вход: n и ориентированные рёбра [u,v]. **Выход:** acyclic и order. При цикле в любой компоненте верните acyclic=false и order=[]. Иначе верните acyclic=true и допустимый порядок всех вершин без повторений, включая изолированные. При n=0 результат acyclic=true и order=[]. Петля является циклом.

Пример: рёбра 0→2, 1→2, 2→3 допускают order=[0,1,2,3] или [1,0,2,3].

Требуемая сложность: O(n+m) времени и памяти.

8. Часть 5: Кратчайшие маршруты доставки

Для ориентированного взвешенного графа и заданного источника найдите кратчайшие пути из источника с помощью обработки DAG в топологическом порядке. Отрицательные веса допустимы.

Реализуйте dag. Проверьте весь граф на наличие циклов, затем обработайте ациклический граф в топологическом порядке, релаксируя исходящие рёбра только из достижимых вершин. Цикл должен быть отклонён, даже если он недостижим из источника.

Вход: n, edges [u,v,weight] и source, где n≥1 и 0≤source<n. **Выход:** acyclic, distance и parent_edge. Для графа с циклом верните acyclic=false и два пустых массива. Для ациклического графа верните acyclic=true и расстояние и родительское ребро для каждой вершины. У источника distance=0 и parent_edge=-1; у недостижимой вершины distance=null

и parent_edge=-1. Для остальных вершин parent_edge указывает на входящее ребро кратчайшего пути из источника. При равных стоимостях принимается любой кратчайший путь.

Пример: n=5, source=0, edges=[[0,1,3],[0,2,8],[1,2,-7],[2,3,4]]. Результат distance=[0,3,-4,0,null] и parent_edge=[-1,0,2,3,-1]. Маршрут к вершине 3: 0→1→2→3, суммарная стоимость 0.

Добавьте вспомогательную функцию восстановления последовательности вершин маршрута по parent_edge. В тестах или анализе продемонстрируйте её работу для достижимой вершины, источника и недостижимой вершины. Параллельные рёбра допустимы; петля делает граф циклическим.

Требуемая сложность: O(n+m) времени и памяти; O(длины маршрута) для восстановления пути.

9. Тестирование и анализ

После отправки изменений откройте **Actions → Public checks** в своём репозитории. Готовая проверка компилирует или запускает выбранную реализацию и выполняет публичные тесты алгоритмов. Для сдаваемого коммита требуется успешный запуск Public checks. Необязательные команды локальной проверки приведены в TESTING.md.

Добавьте собственные случаи в tests/student/, указав вход, ожидаемый и фактический результат. Проверьте применимые случаи: пустые и несвязные графы, изолированные вершины, повторяющиеся рёбра или операции, петли, отрицательные веса, циклы, недостижимые вершины и глубокий обход DFS.

Заполните ANALYSIS.md на английском языке. Для каждого из шести алгоритмов напишите 3–5 предложений об основных шагах, состоянии или инварианте, существенных граничных случаях, временной и пространственной сложности. Объясните сброс состояния между запросами и обработку глубокого DFS.

Сравните Краскала и Прима на одинаковых графах трёх размеров. Укажите n и m, воспроизводимый способ генерации входов, команды или ссылки Actions, не менее трёх замеров каждого алгоритма на каждом размере и медианное время. Проверьте равенство минимальных стоимостей и корректность лесов, затем объясните различия в производительности. Добавьте примеры восстановления маршрута и укажите источники и помощь согласно силлабусу.

После срока сдачи преподаватель проведёт дополнительное тестирование сданного коммита и проверит требуемые реализации и анализ.

10. Сдача работы

В поле **Online text** задания Moodle укажите две ссылки:

1. На коммит GitHub с готовыми реализациями, анализом и собственными тестами. Откройте этот коммит и скопируйте его URL.

2. На успешный запуск **Public checks** для того же коммита. Откройте запуск в Actions и скопируйте его URL.

Завершите сдачу кнопкой **Submit assignment**. Проверяется коммит, указанный в Moodle. При замене работы до срока сдачи обновите обе ссылки. Приватный репозиторий должен оставаться доступным преподавателю.